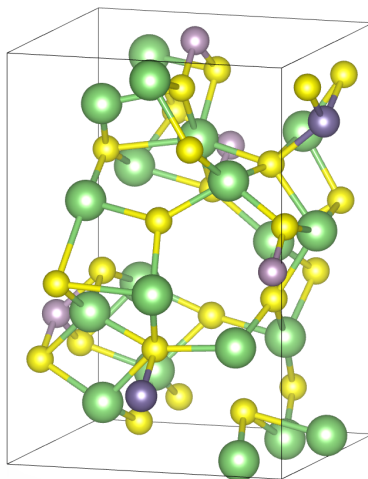


# EnGOAT Example: Li-ion Diffusion in LGPS

In this example, the EnGOAT software is used to study Li-ion diffusion in *lithium germanium phosphorus sulfide* – LGPS ( $\text{Li}_{10}\text{GeP}_2\text{S}_{12}$ ; structure is shown in Figure 1). LGPS is a ceramic material with a remarkable capacity for conducting Li-ions. For this reason, it is widely used as a solid-state electrolyte in Li-ion batteries, improving safety and enabling higher energy density.



**Figure 1:** The crystal structure of LGPS. Yellow atoms represent sulfur, green atoms lithium, light purple phosphorus, and dark purple germanium.

The high Li-ion conductivity of LGPS originates from its crystal structure, which enables relatively free movement of Li-ions along the lattice. The mobility of Li-ions in a crystal lattice is commonly described by the *self-diffusion coefficient*  $D$ . Consequently,  $D$  can be used as a proxy for evaluating the ionic conductivity of solid-state electrolytes. In this example, we demonstrate EnGOAT's ability to efficiently and reliably compute the self-diffusion coefficient of Li-ions in LGPS, along with other metrics, highlighting its applicability for studying existing solid-state electrolytes as well as for high-throughput screening of new candidate materials.

## 1 Preparation

The aim of this example is to run the EnGOAT software using the provided input files located in the `Examples/Li_ions_in_LGPS/Input_files/` directory in the GitHub repository. This section begins with a brief discussion of the input files, followed by a step-by-step guide on how to run the software.

## 1.1 Input Files

Running EnGOAT requires two input files: the simulation parameter file, `input.param`, and the file containing volumetric data, `LGPS.cube`. The structure of both files is described in detail in the *EnGOAT Instruction Manual*, while some practical considerations are provided below.

- `input.param`: The structure of the `input.param` file is presented in Listing 1. Since the goal is to model Li-ion diffusion in LGPS, the choice of the energy step and cutoff must ensure that all relevant energy barriers along diffusion pathways in each direction are captured. If the energy step is larger than the height of the rate-determining energy barriers in the system, these barriers may be missed, leading to an overestimation of diffusion rates. Conversely, if the energy step is too small, a large number of basins separated by negligible energy barriers may be identified, which can slow down the convergence of the kMC simulations. The cutoff energy should be chosen sufficiently high so that configurations at higher energies, which contribute negligibly to diffusion rates, are appropriately excluded. As a rule of thumb for estimating  $D$ , the value of energy step should be set approximately an order of magnitude smaller than the energy at which the first breakthrough is identified. This can be done automatically by turning on the *dynamic E\_step correction* function in the input file (line 4). For this example however, the values of energy step and cutoff of 5 and 35 kJ/mol, respectively, without the dynamic correction, are chosen instead.

Using the chosen energetic parameters, 5 kMC runs with 1000000 steps each yields sufficiently converged diffusion coefficients.

**Listing 1:** The input parameter file `input.param`.

|    |           |                                                           |
|----|-----------|-----------------------------------------------------------|
| 1  | 1         | %Energy unit: 1=kJ/mol, 2=kcal/mol, 3=Ry, 4=eV, 5=Hartree |
| 2  | 5         | %Energy step in kJ/mol                                    |
| 3  | 35        | %Energy cutoff in kJ/mol                                  |
| 4  | 0         | %Dynamic E_step correction? 0=no, 1=yes, 2=stop at break  |
| 5  | 1         | %Run kMC? 0=no, 1=yes                                     |
| 6  | 1         | %Number of temperatures                                   |
| 7  | 1000      |                                                           |
| 8  | 1000000   | %Number of kMC steps to run                               |
| 9  | 5         | %Number of kMC simulations to run for averaging           |
| 10 | 6.941e-3  | %Particle mass                                            |
| 11 | LGPS.cube | %Cube file name                                           |

- `LGPS.cube`: The volumetric data file `LGPS.cube` used in this example contains the *multiparticle* potential energy surface of Li-ions in LGPS and has been generated as described by Gustafsson *et al.* [1]. In addition to the volumetric data, the file also includes information about the crystal structure of LGPS. While this is not strictly required, it is highly useful for visualizing potential energy isosurfaces within the crystal lattice using visualization software such as VESTA. It should be noted that all crystallographic dimensions and atomic coordinates in the `.cube` file must be given in **bohr**.

## 1.2 Running EnGOAT

To run the EnGOAT software with the prepared input files, follow the steps below:

1. Download the EnGOAT/ directory from the GitHub repository.
2. Execute the installation script `install.py` in the EnGOAT/ directory to compile the Fortran subroutines.
3. Place both input files in the EnGOAT/ directory.
4. Run the main script `EnGOAT.py`.

## 2 Result Analysis

If the script is executed correctly, the file `output.log`, along with the directories `MSD_files/`, `NumPy_matrices/`, `Tunnel_data/`, and `TuTraSt_data/`, should be generated within the EnGOAT/ directory.

We begin the analysis by examining the `output.log` file. In the *POTENTIAL ENERGY GRID ANALYSIS* section, presented in Listing 2 below, it can be observed that breakthroughs are recorded in all three crystallographic directions.

**Listing 2:** *POTENTIAL ENERGY GRID ANALYSIS* section of `output.log`.

1 >>> POTENTIAL ENERGY GRID ANALYSIS <<<

2 Number of levels: 7

3 E step: 5.0

4 E cutoff: 35.0

5 -----

6 Level Energy (kJ/mol) Volume (pts) Breakthrough dt(s) Total(s)

7 -----

8 1 5.00 74 / 0.03 0.03

9 2 10.00 400 / 0.09 0.12

10 3 15.00 1388 / 0.06 0.18

11 4 20.00 2976 / 0.11 0.29

12 5 25.00 5004 [0 0 1] 0.11 0.40

13 6 30.00 7226 [0 0 1] 0.17 0.56

14 7 35.00 9212 [1 1 1] 0.17 0.74

15 -----

Further down, in the *TUTRAST ANALYSIS* section (Listing 3), it can be seen that only one three-dimensional tunnel system is identified in the structure. From the difference between the total volume and the accessible volume (which fully belongs to tunnel system 1), it is evident that some basins are not part of the tunnel system and therefore represent non-accessible volume.

These basins can be identified by examining the `basin_data.dat` file located in the `TuTraSt_data/` directory. This information can be used to visualize the tunnel system together with the non-tunnel-forming basins using the grid data stored in the `NumPy_matrices/` directory,

as shown in Figure 2 (right). The Python script used to generate Figure 2 (right) is provided in Appendix A.

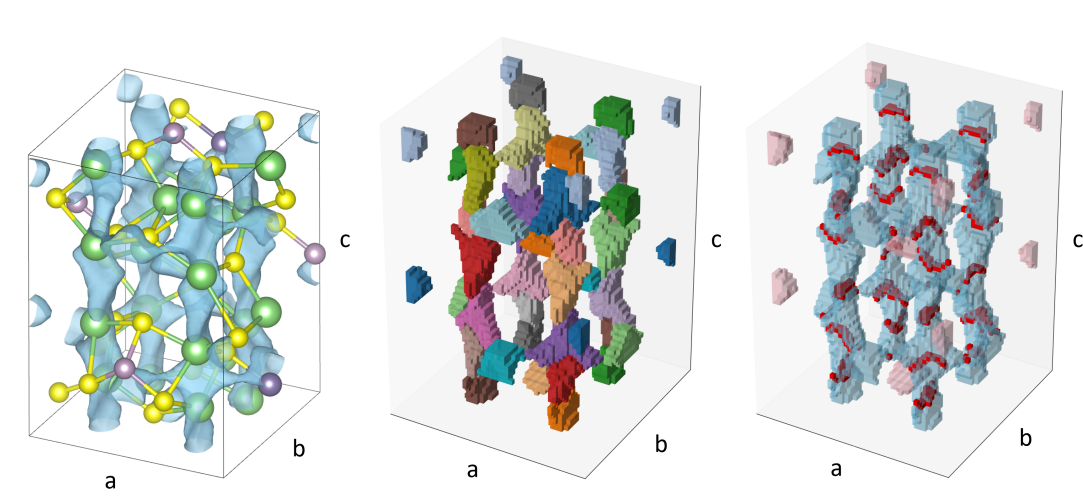
Using the data in the `TuTraSt_data/` directory alongside the grid information in the `NumPy_matrices/` directory enables custom visualization of the system. For example, Figure 2 (middle) was generated in a similar manner, showcasing all basins in the system.

**Listing 3:** *TUTRAST ANALYSIS* section of `output.log`.

```

1 >>> TUTRAST ANALYSIS <<<
2 Total volume:           72.6711 A^3      0.0743 %
3 Accessible volume:      65.1453 A^3      0.0666 %
4 Total surface area:     436.5707 A^2      0.4466 A^2/A^3
5 Accessible surface area: 391.5162 A^2      0.4005 A^2/A^3
6
7 1 tunnel systems were identified:
8 -----
9 Tunnel system #1
10 -----
11 Volume:                 65.1453 A^3      0.0666 %
12 Surface area:           391.5162 A^2      0.4005 A^2/A^3
13 Lowest energy point:     5.8672 kJ/mol
14 Dimensionality:          3

```



**Figure 2:** The PES landscape of Li-ions in LGPS. LGPS structure together with the Li-ion potential energy isosurface at 35 kJ/mol (left), Individual basins identified by the flooding algorithm (middle), and transition-state surfaces between the basins (right). On the right panel, the tunnel-forming basins belonging to tunnel system 1 are colored blue, the transition-state surfaces between them dark red, and the non-tunnel-forming basins (basins that are not connected into a tunnel system) are colored pink.

Finally, in the *RUNNING kMC SIMULATIONS* section (Listing 4), the calculated self-diffusion coefficients are reported. This section begins with tables showing the self-diffusion coefficient of a Li-ion in each tunnel system (in this case, only one) for each individual kMC run.

The MSD files generated during the kMC simulations are stored in the `MSD_files/` directory and can be visualized using the `MSD_plotter.py` tool located in the `tools/` directory of the GitHub repository. To generate the MSD plots, download the tool, place it in the `EnGOAT/`

directory, and execute it. The script will automatically create a directory named `MSD_plots/`, where the plots are saved as `.png` files. The MSD plots for this example are shown in Figure 3.

**Listing 4: *RUNNING kMC SIMULATIONS* section of `output.log`.**

```

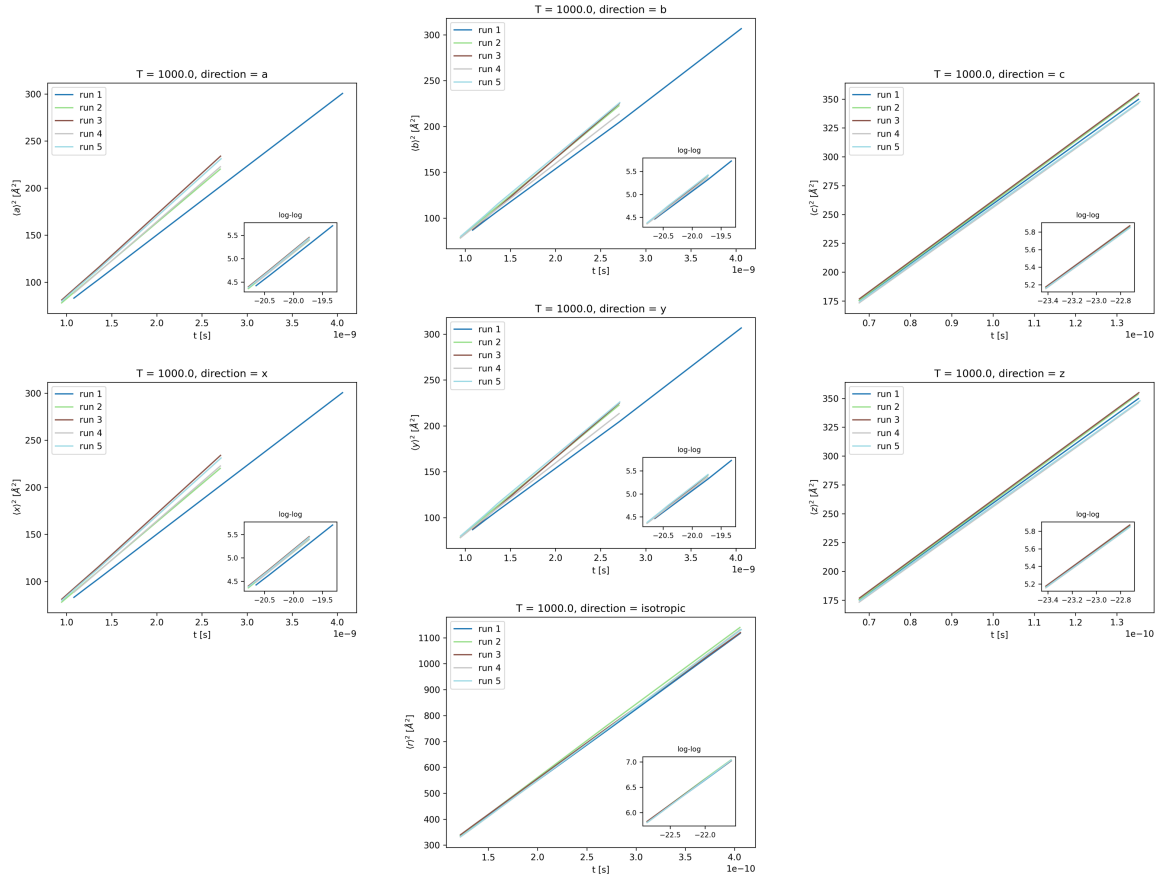
1 >>> RUNNING kMC SIMULATIONS (T = 1000.0 K) <<<
2 Number of runs: 5
3 Number of steps: 1000000
4
5 Tunnel system 1, weight = 0.7963145169643064
6 -----
7 Run D_a (m^2/s) D_b (m^2/s) D_c (m^2/s) dt (s) total (s)
8 -----
9 1 4.102e-06 4.234e-06 1.302e-04 2.82 2.82
10 2 3.607e-06 3.654e-06 1.290e-04 2.79 5.60
11 3 4.474e-06 4.341e-06 1.327e-04 2.78 8.38
12 4 4.267e-06 4.304e-06 1.277e-04 2.77 11.16
13 5 4.167e-06 4.080e-06 1.292e-04 2.75 13.90
14 -----
15 Average self diffusion coefficient in a b c:
16 D_a = 4.123e-06+-2.87e-07 D_b = 4.122e-06+-2.51e-07 D_c = 1.297e-04+-1.67e-06
17 Average self diffusion coefficient in x y z:
18 D_x = 4.123e-06+-2.87e-07 D_y = 4.122e-06+-2.51e-07 D_z = 1.297e-04+-1.67e-06
19 Average Isotropic self diffusion coefficient:
20 D = 4.602e-05+-1.07e-06
21 -----
22 Completed kMC simulations for tunnel system 1 at T = 1000.0 K.
23 Total time elapsed: 13.93 s
24
25 -----
26 Joint diffusion coefficients across all tunnel systems at T = 1000.0 K:
27 In the a b c directions:
28 D_a = 3.284e-06+-2.29e-07 D_b = 3.283e-06+-2.00e-07 D_c = 1.033e-04+-1.33e-06
29 In the x y z directions:
30 D_x = 3.284e-06+-2.29e-07 D_y = 3.283e-06+-2.00e-07 D_z = 1.033e-04+-1.33e-06
31 Isotropic self diffusion coefficient:
32 D = 3.665e-05+-8.51e-07

```

At the top of each table, the tunnel system *weight* is displayed. This is defined as the ratio of the Boltzmann-weighted volume of a given tunnel system to that of the entire system, and thus represents the probability of finding a Li-ion within that tunnel system. In this example, the probability of a Li-ion occupying the tunnel system is 80%, while the remaining 20% corresponds to occupation of the non-tunnel-forming basins.

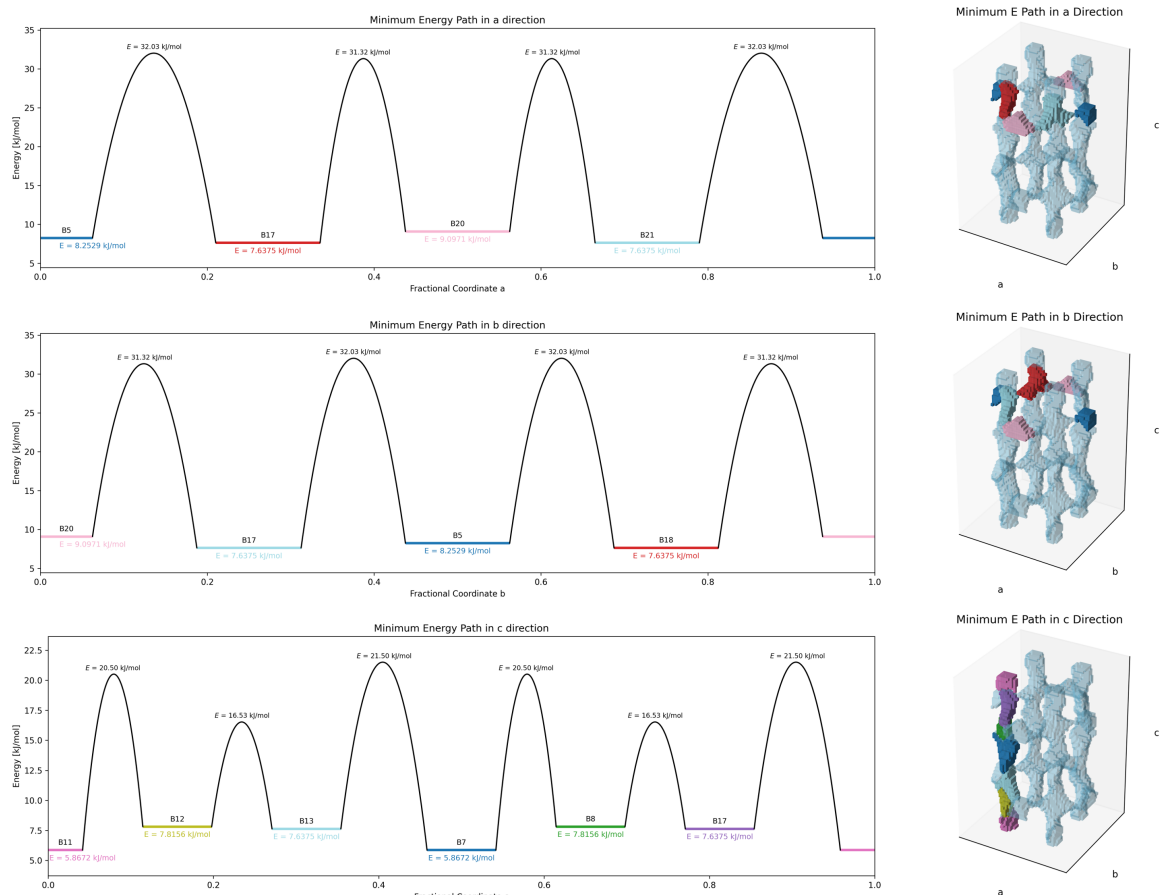
The joint diffusion coefficient across all tunnel systems (calculated as a weighted sum) at a given temperature is reported at the end of the section (lines 26–32). It can be observed that diffusion in the *c* direction is two orders of magnitude higher than in the *a* and *b* directions. This difference arises from lower energy barriers for Li-ion movement along the *c* direction.

To visualize the minimum energy path along with the associated energy barriers in a given direction, the Minimum E Pathway `plotter.py` tool (located in the `tools/` directory of the GitHub repository) can be used. Simply download the tool, place it in the `EnGOAT/` directory, and



**Figure 3:** MSD curves for the example of Li-ions in LGPS along the crystallographic directions (top), cartesian directions (middle), and the isotropic MSD curves (bottom).

execute it. The script will generate a directory named `Min E path plots/`, where the minimum energy pathway plots and corresponding path visualizations are saved. The figures generated for this example are shown in Figure 4.



**Figure 4:** he minimum-energy pathways of Li-ions in LGPS along *a* (top), *b* (middle), and *c* (bottom) directions. In the energy profiles (left), individual basins are shown as colored horizontal lines. Each basin is labeled with its basin identifier (BID) above the line and its lowest energy value below. The position of the center of each line corresponds to the fractional coordinate of that basin along the given crystallographic direction. Transitions between basins are represented by black connecting lines, with the corresponding transition-state energies indicated above them. The images on the right show the structure of the given tunnel system (show in transparent blue), with the basins involved in the given minimum-energy pathway highlighted with the same colors as in the corresponding energy plots.

### 3 References

- (1) Gustafsson, H.; Kozdra, M.; Smit, B.; Barthel, S.; Mace, A. Predicting ion diffusion from the shape of potential energy landscapes. *Journal of Chemical Theory and Computation* **2023**, *20*, 18–29.

## A Python Script for Figure 2 (right)

```
import numpy as np
import matplotlib.pyplot as plt

# Load the numpy matrices
Basin_matrix = np.load("Basin_matrix.npy")
TS_matrix = np.load("TS_matrix.npy")
Level_matrix = np.load("Level_matrix.npy")
Tunnel_matrix = np.load("Tunnel_matrix.npy")

# Create a figure and 3D axes
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

mask = Tunnel_matrix == 1
ax.voxels(mask, facecolors="skyblue", edgecolor='none', alpha=0.4)

mask = Basin_matrix == 1
ax.voxels(mask, facecolors="pink", edgecolor='none', alpha=0.4)

mask = Basin_matrix == 2
ax.voxels(mask, facecolors="pink", edgecolor='none', alpha=0.4)

mask = Basin_matrix == 3
ax.voxels(mask, facecolors="pink", edgecolor='none', alpha=0.4)

mask = Basin_matrix == 4
ax.voxels(mask, facecolors="pink", edgecolor='none', alpha=0.4)

mask = Basin_matrix == -1
ax.voxels(mask, facecolors="red", edgecolor='none', alpha=0.8)

#Configure the plot
ax.set_xlabel("a")
ax.set_ylabel("b")
ax.set_zlabel("c")
ax.set_title(f"Minimum E path {dir}")

ax.grid(False)

ax.set_xticks([])
ax.set_yticks([])
ax.set_zticks([])

ax.set_box_aspect([1,1,1])

plt.savefig(f"direction {dir}.png")
plt.show()
plt.close()
```